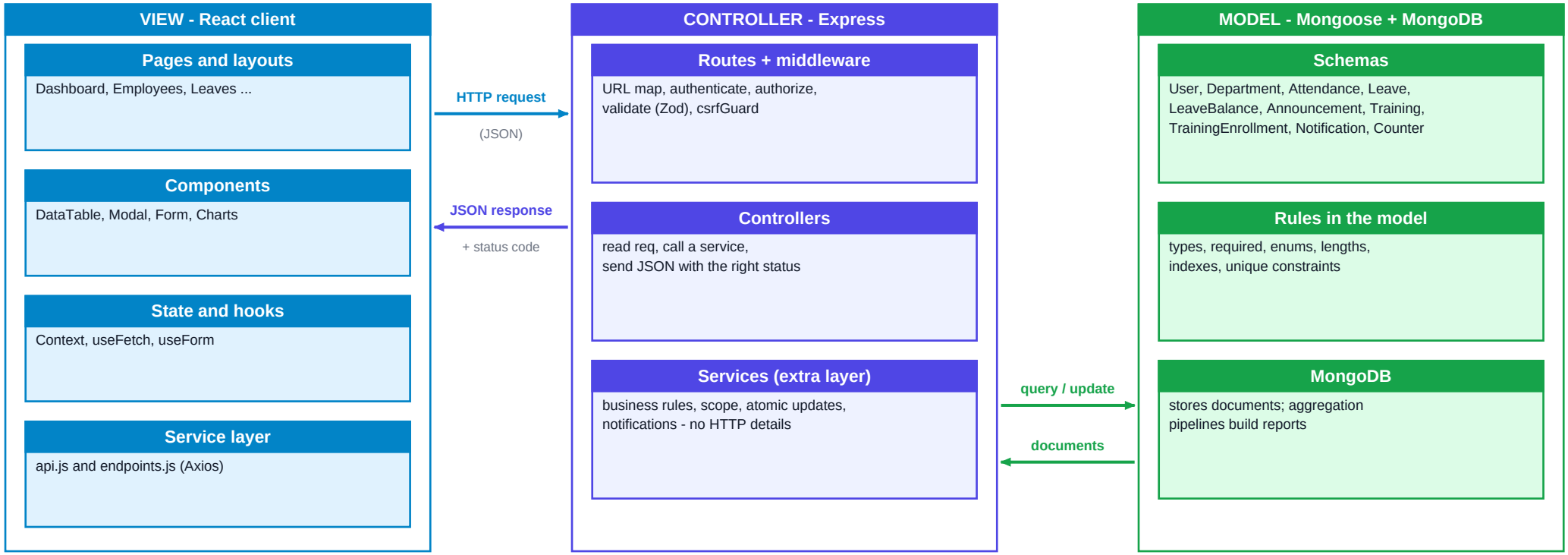
MVC Pattern in StaffSync

Model - View - Controller, plus a Service layer for business rules

MVC splits an application into three parts so that data, screens and request handling can change independently. In a MERN project the parts live in two programs: the **View** is the React app in the browser, and the **Controller** and **Model** live in the Express and MongoDB server. They talk through JSON over HTTP, so StaffSync does not render HTML on the server.



Rule of thumb: controllers stay thin, services hold the rules, models describe the data, the view only displays and asks.

Where each MVC part lives

Folder-to-role mapping with real files from the project

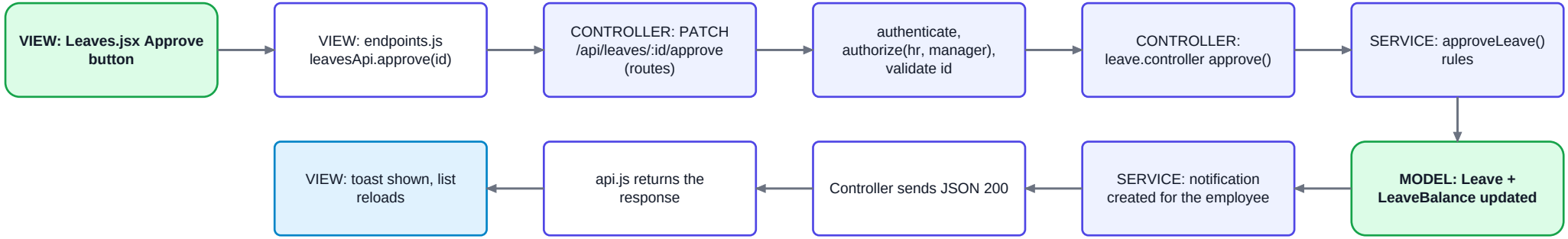
MVC part	Folder	Example files	Responsibility	Must NOT contain
Model	server/src/models	User.js, Leave.js, LeaveBalance.js, Attendance.js, Training.js	Shape of the data: fields, types, enums, defaults, indexes, unique constraints, hidden fields.	HTTP code, business workflows
View	client/src/pages, components, layouts, features	Leaves.jsx, Employees.jsx, DataTable.jsx, Modal.jsx, HRDashboard.jsx	What the user sees and clicks; local form state; shows loading, error and empty states.	Direct database access, secrets, security decisions
Controller (routing)	server/src/routes, middleware, validators	leave.routes.js, auth.js, csrf.js, validate.js, leave.validators.js	Maps URL + method to a handler; runs authentication, role check and input validation in order.	Business rules
Controller (handlers)	server/src/controllers	leave.controller.js, user.controller.js	Takes validated input, calls one service function, returns { success, data, meta } with a status code.	Database queries, multi-step logic
Service layer (extra)	server/src/services	leave.service.js, attendance.service.js, scope.js, report.service.js	Business rules: team scope, overlap and balance checks, atomic claim-then-deduct, notifications.	req and res objects
Client service layer	client/src/services	api.js, endpoints.js	One function per API call; adds the cookie, CSRF header and 401 handling.	JSX or UI logic

Why add a Service layer? Classic MVC leaves "where do the rules go?" open. Putting them in services keeps controllers short, lets dashboards and tests reuse the same rules, and keeps Express objects (req, res) out of business code.

MVC across a network: because the View is a separate single-page app, the controller returns JSON instead of a rendered page. The same API could serve a mobile app without changing models or services.

Example: a manager approves a leave request

One click, followed through every layer



The same path in code (shortened)

```
// ROUTE   server/src/routes/leave.routes.js
r.patch('/:id/approve', authorize('hr', 'manager'), validate(idParam, 'params'), c.approve);

// CONTROLLER   server/src/controllers/leave.controller.js
export const approve = asyncHandler(async (req, res) =>
  ok(res, await leaves.approveLeave(req.user, req.params.id), 'Leave approved'));

// SERVICE   server/src/services/leave.service.js  (rules live here)
const leave = await loadForReview(actor, id);       // not own request, direct report or HR, still pending
const claimed = await Leave.findOneAndUpdate(        // atomic: only one reviewer can win
  { _id: id, status: 'pending' }, { status: 'approved', reviewedBy: actor._id }, { new: true });
const deducted = await LeaveBalance.findOneAndUpdate( // deduct only if enough days remain
  { user: leave.user, year: leave.year, type: leave.type,
    $expr: { $lte: [{ $add: ['$used', leave.days] }, '$allocated'] } }, { $inc: { used: leave.days } });

// MODEL   server/src/models/Leave.js
status: { type: String, enum: LEAVE_STATUSES, default: 'pending' }
```

Benefits, rules and interview notes

Why StaffSync is organised this way

Why separate the layers

- **Change one thing at a time:** restyle the View without touching rules; change a rule in a service without touching routes.
- **Test each layer:** API tests exercise routes, services and models together with Supertest; component tests exercise the View.
- **Security in the right place:** access control sits in middleware and services on the server, never only in the View.
- **Reuse:** dashboards, reports and controllers call the same services, so a rule is written once.

Do and do not

Do	Do not
Validate every input with Zod in the route chain.	Trust ids or roles sent by the browser.
Keep controllers to: read input, call a service, send a response.	Put queries or if-else rule chains in controllers.
Keep schema rules (types, unique, enum) in models.	Rely on the View to enforce permissions.
Call APIs only through client/src/services.	Call Axios from inside page components.

Likely interview questions

Question	Short answer
What is MVC and how did you apply it?	It separates data (Model), user interface (View) and request handling (Controller). In StaffSync the Model is Mongoose schemas, the View is the React app, and the Controller is Express routes and controllers, with a service layer holding the business rules.
Why a service layer on top of MVC?	Rules such as claim-then-deduct leave approval need to be reused and tested without HTTP objects, and it keeps controllers thin.
Is React the "V" in MVC?	Yes, it renders the interface. State management and the API service layer sit beside it; the real Controller and Model are on the server.
Where is authorization enforced and why?	In route middleware and in services (scope rules), because the browser can be modified by the user. The React route guard only improves the experience.
How would you add a new feature, for example Holidays?	Model: Holiday schema. Validator: Zod schema. Service: rules. Controller and route: endpoints with authorize(hr). View: page, endpoint functions and a menu item.